

---

# Databricks Certified Data Engineer Associate

Fiche de revision complete

---

Version de l'examen : **4 mai 2026**

Basee sur le guide officiel Databricks + AI Prep Guide

## En bref

|                         |                                        |
|-------------------------|----------------------------------------|
| <b>Questions notees</b> | 45 QCM                                 |
| <b>Duree</b>            | 90 minutes                             |
| <b>Prix</b>             | 200 USD (+ taxes)                      |
| <b>Format</b>           | En ligne ou centre de test             |
| <b>Aides</b>            | Aucune autorisee                       |
| <b>Prerequis</b>        | Aucun (6 mois de pratique recommandes) |
| <b>Validite</b>         | 2 ans (recertification requise)        |

## Table des matières

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Carte des priorites</b>                                           | <b>2</b>  |
| <b>1 Pieges de renommage (a lire en premier)</b>                     | <b>2</b>  |
| <b>2 Databricks Intelligence Platform (6%)</b>                       | <b>2</b>  |
| 2.1 Architecture en couches . . . . .                                | 3         |
| 2.2 Delta Lake . . . . .                                             | 3         |
| 2.3 Unity Catalog (UC) . . . . .                                     | 3         |
| 2.4 Services de compute (clusters) . . . . .                         | 3         |
| <b>3 Data Ingestion and Loading (21%)</b>                            | <b>4</b>  |
| 3.1 Les patterns d'ingestion . . . . .                               | 4         |
| 3.2 COPY INTO . . . . .                                              | 4         |
| 3.3 Auto Loader (cloudFiles) . . . . .                               | 4         |
| 3.4 COPY INTO vs Auto Loader : le choix cle . . . . .                | 4         |
| 3.5 Lakeflow Connect . . . . .                                       | 5         |
| <b>4 Data Transformation and Modeling (22%)</b>                      | <b>5</b>  |
| 4.1 Architecture medaillon . . . . .                                 | 5         |
| 4.2 Nettoyage Bronze → Silver . . . . .                              | 5         |
| 4.3 Jointures (joins) . . . . .                                      | 6         |
| 4.4 Manipulations de colonnes/lignes . . . . .                       | 6         |
| 4.5 Parametres de tuning a connaitre . . . . .                       | 6         |
| 4.6 Objets de la couche Gold (UC) . . . . .                          | 6         |
| <b>5 Working with Lakeflow Jobs (16%)</b>                            | <b>7</b>  |
| 5.1 Orchestration par DAG . . . . .                                  | 7         |
| 5.2 Control flow . . . . .                                           | 7         |
| 5.3 Triggers (declencheurs) . . . . .                                | 7         |
| <b>6 Implementing CI/CD (10%)</b>                                    | <b>7</b>  |
| 6.1 Git Folders (ex-Repo) . . . . .                                  | 7         |
| 6.2 Automation Bundles (ex-Databricks Asset Bundles / DAB) . . . . . | 8         |
| 6.3 Databricks CLI . . . . .                                         | 8         |
| <b>7 Troubleshooting, Monitoring, Optimization (10%)</b>             | <b>8</b>  |
| 7.1 Monitoring via l'UI Lakeflow Jobs . . . . .                      | 8         |
| 7.2 Bottlenecks dans la Spark UI . . . . .                           | 9         |
| 7.3 Optimisations Delta . . . . .                                    | 9         |
| 7.4 Diagnostics cluster . . . . .                                    | 9         |
| <b>8 Governance and Security (15%)</b>                               | <b>9</b>  |
| 8.1 Tables managed vs external . . . . .                             | 9         |
| 8.2 Controle d'accès : GRANT / REVOKE / DENY . . . . .               | 10        |
| 8.3 Masquage et securite au niveau ligne/colonne . . . . .           | 10        |
| <b>9 Checklist hands-on (a savoir faire les yeux fermes)</b>         | <b>10</b> |
| <b>10 Les 5 questions officielles du guide (corrigees)</b>           | <b>11</b> |
| <b>Annexe : memo derniere minute</b>                                 | <b>11</b> |

## Carte des priorites : ponderation par domaine

Le temps de revision doit suivre le poids de chaque domaine. **Transformation (22%) et Ingestion (21%) representent a eux seuls 43% de l'examen** : c'est la ou il faut investir le plus.

| Domaine                                  | Poids       | ≈ questions |
|------------------------------------------|-------------|-------------|
| §3 – Data Transformation and Modeling    | <b>22%</b>  | ~10         |
| §2 – Data Ingestion and Loading          | <b>21%</b>  | ~9          |
| §4 – Working with Lakeflow Jobs          | 16%         | ~7          |
| §7 – Governance and Security             | 15%         | ~7          |
| §5 – Implementing CI/CD                  | 10%         | ~5          |
| §6 – Troubleshooting, Monitoring, Optim. | 10%         | ~5          |
| §1 – Databricks Intelligence Platform    | 6%          | ~3          |
| <b>Total</b>                             | <b>100%</b> | <b>45</b>   |

### 1. Pieges de renommage (a lire en premier)

Les modeles d'IA ont ete entraines *avant* les changements recents de la plateforme Databricks : ils utilisent souvent les anciens noms. **L'examen utilise les noms actuels**. Memorise ce tableau – c'est la cause n°1 d'echec en preparation avec l'IA.

| Ancien nom                     | Nom actuel (2026)                           | Role                                       |
|--------------------------------|---------------------------------------------|--------------------------------------------|
| Databricks Workflows           | <b>Lakeflow Jobs</b>                        | Orchestration de pipelines (DAG de taches) |
| Delta Live Tables (DLT)        | <b>Lakeflow Spark Declarative Pipelines</b> | Pipelines declaratifs ETL                  |
| Databricks Repos               | <b>Git Folders</b>                          | Integration Git dans le workspace          |
| Databricks Asset Bundles (DAB) | <b>Automation Bundles (Declarative)</b>     | Packaging / promotion dev-test-prod        |
| Partner Connect / connecteurs  | <b>Lakeflow Connect</b>                     | Ingestion managee depuis sources externes  |

#### A retenir

Si une reponse de QCM emploie un ancien nom (Workflows, DLT, Repos) alors qu'une autre emploie le nom actuel pour la meme chose, l'examen tend a privilegier la terminologie **a jour**. Mais attention : le concept prime sur le nom – le fond ne change pas.

### 2. Databricks Intelligence Platform (6%)

## 2.1 Architecture en couches

La plateforme separe **control plane** (gere par Databricks : UI, orchestration, metadonnees) et **compute/data plane** (dans ton cloud : clusters, stockage objet ADLS/S3/GCS).

## 2.2 Delta Lake

Format de table par default. Points cles pour l'examen :

- **Transactions ACID** sur le stockage objet (via le transaction log `_delta_log`).
- **Time travel** : interroger une version anterieure (`VERSION AS OF, TIMESTAMP AS OF`) → rollback fiable apres un mauvais ingest.
- **Schema enforcement + schema evolution**.
- **MERGE** (upsert), **OPTIMIZE**, **VACUUM**.

## 2.3 Unity Catalog (UC)

Couche de gouvernance centralisee. Hierarchie a 3 niveaux :

catalog → schema (database) → table / view / volume / model

UC fournit : controle d'accès fin, lineage, audit, partage. **Source unique de verite** pour l'IA et le BI.

## 2.4 Services de compute (clusters)

| Type          | Usage / caracteristiques                                              |
|---------------|-----------------------------------------------------------------------|
| All-purpose   | Developpement interactif, notebooks partages, exploration. Plus cher. |
| Job cluster   | Cree pour un job, detruit a la fin. Moins cher pour l'ETL planifie.   |
| SQL Warehouse | Requetes SQL/BI, demarrage rapide, multi-utilisateurs concurrents.    |
| Single-node   | Petits tests, dev leger, pas de workers.                              |

### ! Piege d'examen

Pour **plusieurs analystes lancant des requetes SQL ad hoc** avec demarrage rapide et concurrence : la bonne reponse penche vers un **SQL Warehouse** (ou cluster high-concurrency avec autoscaling) – *pas* un job cluster (fait pour l'ETL planifie) ni un single-node.

### QCM

Un ingenieur veut : iteration rapide, rollback fiable apres mauvais ingest, piste d'audit reglementaire, et source unique de verite pour IA + BI. Quelle strategie ?

A. Stockage CSV sur DBFS + copies nocturnes      B. **Delta Lake (ACID + time travel) gouverne par Unity Catalog**

C. Stockage objet seul + requetes SQL ad hoc      D. DataFrames en memoire ephemerer

*Reponse : B. Delta = ACID + time travel (rollback, audit); UC = acces coherent et lineage. Les autres options n'offrent ni transactions ni gouvernance.*

### 3. Data Ingestion and Loading (21%)

#### 3.1 Les patterns d'ingestion

Batch / streaming / incremental. Le choix depend de : volume, frequence, types de donnees, besoins de gouvernance.

#### 3.2 COPY INTO

Commande SQL **idempotente** : charge de maniere incrementale des fichiers depuis le stockage objet vers une table UC. Retient les fichiers deja charges → pas de doublons.

```
1 COPY INTO mon_catalogue.bronze.ventes
2 FROM 's3://bucket/ventes/'
3 FILEFORMAT = CSV
4 FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
5 COPY_OPTIONS ('mergeSchema' = 'true');
```

Listing 1 – COPY INTO depuis le stockage objet

#### 3.3 Auto Loader (cloudFiles)

Ingestion **incrementale en streaming** de nouveaux fichiers. Gere schema inference, schema evolution et checkpointing. Deux modes de detection :

- **Directory listing** : liste le repertoire (simple, par default).
- **File notification** : s'abonne aux evenements du cloud (scalable, gros volumes).

```
1 df = (spark.readStream
2     .format("cloudFiles")
3     .option("cloudFiles.format", "json")
4     .option("cloudFiles.schemaLocation", "/chk/schema")
5     .load("s3://bucket/entrant/"))
6
7 (df.writeStream
8     .option("checkpointLocation", "/chk/ventes")
9     .option("mergeSchema", "true")
10    .trigger(availableNow=True) # mode batch : traite tout puis s'arrete
11    .toTable("mon_catalogue.bronze.evenements"))
```

Listing 2 – Auto Loader avec evolution de schema

#### 3.4 COPY INTO vs Auto Loader : le choix cle

|                             | COPY INTO                          | Auto Loader                            |
|-----------------------------|------------------------------------|----------------------------------------|
| Nature                      | SQL, batch                         | Structured Streaming                   |
| Volume de fichiers          | Milliers (modere)                  | Millions (tres scalable)               |
| Detection nouveaux fichiers | Suivi interne des fichiers charges | Directory listing ou file notification |
| Schema evolution            | mergeSchema                        | Automatique + schemaLocation           |
| Ideal pour                  | Chargements periodiques simples    | Flux continu / gros volumes            |

### 3.5 Lakeflow Connect

Connecteurs **managed** (sources SaaS/BD d'entreprise : Salesforce, SQL Server...) et **standard**. Ingestion fiable et gouvernee dans des tables UC, y compris donnees semi-structurees (JSON, nested).

#### ! Piège d'examen

**Auto Loader** = gros volumes, streaming, evolution de schema auto. **COPY INTO** = batch SQL idempotent, volumes moderes. **Lakeflow Connect** = sources d'entreprise managees. Ne les confonds pas : le choix se justifie par volume + frequence + type de source.

#### ! Piège d'examen : logs d'audit

Les **audit logs** Databricks sont livres en **JSON**, generalement sous 15 min, et de nouvelles livraisons **peuvent ecraser** des fichiers existants (ce n'est pas CSV, pas Parquet, pas hebdomadaire).

#### QCM

Besoin : ingerer en continu des millions de petits fichiers JSON arrivant en permanence, avec evolution de schema automatique. Quelle methode ?

- A. COPY INTO en batch nocturne                      B. **Auto Loader (cloudFiles) en streaming**  
C. INSERT manuel par notebook                      D. Lakeflow Connect vers Salesforce

Reponse : **B. Auto Loader gere volume massif + schema evolution + checkpointing.**

## 4. Data Transformation and Modeling (22%)

### 4.1 Architecture medaillon

| Couche | Role                                                                             |
|--------|----------------------------------------------------------------------------------|
| Bronze | Donnees brutes ingerees telles quelles (append).                                 |
| Silver | Donnees nettoyees : nulls traitees, types standardises, dedupliees, jointures.   |
| Gold   | Donnees agregees pretes pour le BI/analytics (tables, vues, vues materialisees). |

### 4.2 Nettoyage Bronze → Silver

```

1 from pyspark.sql import functions as F
2
3 bronze = spark.read.table("cat.bronze.clients")
4
5 silver = (bronze
6     .na.drop(subset=["client_id"])                # supprimer nulls cles
7     .withColumn("date", F.to_date("date_str"))    # standardiser le type
8     .dropDuplicates(["client_id"]))               # deduplication
9
10 silver.write.mode("overwrite").saveAsTable("cat.silver.clients")

```

Listing 3 – Lire bronze, nettoyer, ecrire silver

### 4.3 Jointures (joins)

- `inner`, `left`, `right`, `full`, `cross`.
- **Broadcast join** : diffuse la petite table sur tous les executors → évite un shuffle coûteux. Contrôle par `spark.sql.autoBroadcastJoinThreshold`.
- Jointure sur **cles multiples**, puis `union` / `unionAll`.

```

1 from pyspark.sql.functions import broadcast, approx_count_distinct, mean
2
3 df = grandes.join(broadcast(petites), "cle", "left")
4
5 agg = (df.groupBy("region")
6         .agg(approx_count_distinct("client_id").alias("clients"),
7             mean("montant").alias("panier_moyen")))

```

Listing 4 – Broadcast join et agregations

### 4.4 Manipulations de colonnes/lignes

Ajouter/supprimer/renommer colonnes, filtres, split, **exploder des tableaux** (`explode`), deduplication, agregations (`count`, `approx_count_distinct`, `mean`, `summary`).

### 4.5 Parametres de tuning a connaitre

| Parametre                                                        | Effet                                                 |
|------------------------------------------------------------------|-------------------------------------------------------|
| <code>spark.sql.shuffle.partitions</code>                        | Nombre de partitions apres un shuffle (join/groupBy). |
| <code>spark.default.parallelism</code>                           | Parallelisme par default des RDD.                     |
| <code>spark.executor.memory</code><br><code>driver.memory</code> | / Memoire allouee par executor / driver.              |
| <code>spark.sql.autoBroadcastJoinThreshold</code>                | Seuil sous lequel une table est broadcastee.          |

### 4.6 Objets de la couche Gold (UC)

| Objet             | Description                                                              |
|-------------------|--------------------------------------------------------------------------|
| Table (managed)   | Donnees + metadonnees geres par UC.                                      |
| View              | Requete sauvegardee, calculee a la lecture (pas de stockage).            |
| Materialized view | Resultat precalcule et rafraichi → requetes BI rapides.                  |
| Streaming table   | Table alimentee en continu par un flux (ingestion/transfo incrementale). |

#### ! Piège d'examen

**View** = recalculée à chaque lecture (toujours fraîche, plus lente). **Materialized view** = précalculée (rapide, mais doit être rafraîchie). **Streaming table** = mise à jour incrémentale continue. Sais distinguer les trois usages.

#### QCM

Une équipe BI veut des tableaux de bord rapides sur des agrégats coûteux recalculés souvent. Quel objet Gold ?

A. Vue standard      B. **Vue materialisee**      C. Table temporaire      D. CTE

Reponse : **B.** La vue materialisee precalcule et stocke le resultat → lectures BI rapides, contrairement a la vue standard recalculée a chaque fois.

## 5. Working with Lakeflow Jobs (16%)

### 5.1 Orchestration par DAG

Un **Lakeflow Job** (ex-Workflow) est un graphe orienté acyclique (DAG) de **taches** avec dépendances. Types de taches : **notebook**, **SQL query**, **dashboard**, **pipeline** (Lakeflow Spark Declarative Pipeline).

### 5.2 Control flow

- **Retries** : reessais automatiques en cas d'échec.
- **Conditional / branching** : tache If/else selon une condition.
- **Looping** : For each sur une liste de valeurs.
- **Dependances** : une tache s'exécute apres la réussite de ses parents.

### 5.3 Triggers (declencheurs)

| Trigger          | Declenche quand...                                             |
|------------------|----------------------------------------------------------------|
| Scheduled (cron) | Base sur le temps : a intervalle fixe (ex. toutes les heures). |
| File arrival     | Un nouveau fichier arrive a un emplacement de stockage.        |
| Table update     | Une table amont est mise a jour (data-driven).                 |

#### ! Piège d'examen

**Time-based** (scheduled) : pour des données a horaire previsible. **Data-driven** (file arrival / table update) : pour reagir a la *disponibilite réelle* des données, sans attendre une heure fixe ni interroger inutilement.

#### QCM

Les données sources arrivent a des heures imprevisibles. On veut lancer le pipeline des qu'un fichier est depose, sans polling. Quel trigger ?

A. Scheduled toutes les 5 min B. **File arrival** C. Manuel D. Table update sur une table sans lien

Reponse : **B.** Le trigger file arrival reagit a l'arrivee du fichier – data-driven, plus efficace qu'un cron serre.

## 6. Implementing CI/CD (10%)

### 6.1 Git Folders (ex-Repos)

Dans le workspace : creer/changer de branche, commit, push, ouvrir une pull request via l'integration Git. Workflow de dev versionne.



## 6.2 Automation Bundles (ex-Databricks Asset Bundles / DAB)

Format déclaratif (`databricks.yml`) qui **package et promeut** du code + des ressources (Lakeflow Jobs, pipelines déclaratifs, etc.) à travers **dev / test / prod**. Gestion des différences d'environnement par **variables et overrides** (targets).

```

1 bundle:
2   name: mon_pipeline_etl
3
4 targets:
5   dev:
6     default: true
7     workspace: { host: https://dev.cloud.databricks.com }
8   prod:
9     workspace: { host: https://prod.cloud.databricks.com }
10    variables: { catalogue: prod_cat }
```

Listing 5 – Structure d'un bundle (`databricks.yml`, simplifié)

## 6.3 Databricks CLI

Valide, déploie et gère les bundles dans des workflows CI/CD automatisés :

- `databricks bundle validate` – vérifie la config.
- `databricks bundle deploy -t prod` – déploie sur la cible.
- `databricks bundle run` – lance un job du bundle.

### ! Piège d'examen

Pour **deployer, versionner et orchestrer** des pipelines ETL de façon modulaire et répétable (CI/CD) : la bonne réponse est **Automation Bundles + Git + CLI**, *pas* des wheels dans des Volumes, ni la revision history des notebooks comme système de versioning.

### QCM

Une équipe veut un moyen modulaire de déployer, versionner et orchestrer ses pipelines ETL avec CI/CD et répétabilité. Quelle fonctionnalité ?

- |                                             |                                     |
|---------------------------------------------|-------------------------------------|
| A. Modèles UC pour représenter les jobs ETL | B. Wheels dans des Volumes UC       |
| C. Notebook monte sur Volume + Jobs API v2  | D. <b>Automation Bundles (DABs)</b> |
- versionnées dans Git, promues par CI/CD**

*Reponse : D. Les bundles définissent ressources + code, versionnées dans Git, promues dev→test→prod par CI/CD automatisée.*

## 7. Troubleshooting, Monitoring, Optimization (10%)

### 7.1 Monitoring via l'UI Lakeflow Jobs

- **Run history** : comparer les temps d'exécution actuels aux baselines historiques (détecter les tendances de dégradation).
- Interpréter les statuts de jobs, lire le **DAG** pour repérer les blocages amont.
- Suivre les temps d'exécution et les taux d'échec.

## 7.2 Bottlenecks dans la Spark UI

| Probleme      | Symptome / metrique                                                            |
|---------------|--------------------------------------------------------------------------------|
| Data skew     | Une tache beaucoup plus longue que les autres ; Max shuffle read $\gg$ Median. |
| Shuffling     | Beaucoup d'echange de donnees entre executors (joins/groupBy).                 |
| Disk spilling | Donnees ecrites sur disque faute de memoire (spill memory/disk).               |

### ! Piege d'examen : data skew

Symptome typique : *la plupart des taches finissent en <30s mais une prend >10 min, Max shuffle read  $\gg$  Median.* C'est du **data skew**. La solution robuste : verifier que l'**Adaptive Query Execution (AQE) avec skew join handling** est actif (il decoupe la partition surdimensionnee au runtime). Ajouter des executors ne corrige pas le desequilibre.

## 7.3 Optimisations Delta

- **Liquid Clustering** : remplace le partitionnement + ZORDER, s'adapte automatiquement aux patterns de requetes (CLUSTER BY).
- **Predictive Optimization** : Databricks execute automatiquement OPTIMIZE et VACUUM au bon moment, sans reglage manuel.

## 7.4 Diagnostics cluster

Savoir diagnostiquer : echecs de demarrage de cluster, conflits de librairies, erreurs out-of-memory (OOM).

### QCM

Dans la Spark UI, une tache prend 10 min alors que les autres finissent en 30s, avec un Max shuffle read a 5 GB contre 400 MB en median. Que faire ?

- A. Agrandir le cluster      B. **Confirmer que l'AQE avec skew join handling est actif**  
 C. Reduire `shuffle.partitions`      D. Repartitionner manuellement avec une salt key

*Reponse : B. C'est du data skew ; l'AQE decoupe automatiquement la partition surdimensionnee. (D peut aider mais B est la solution native recommandee.)*

## 8. Governance and Security (15%)

### 8.1 Tables managed vs external

|            | Managed                              | External                                  |
|------------|--------------------------------------|-------------------------------------------|
| Donnees    | Gerees par UC (emplacement gere)     | A un emplacement externe que tu specifies |
| DROP TABLE | Supprime donnees + metadonnees       | Supprime seulement les metadonnees        |
| Usage      | Cas par default, cycle de vie simple | Donnees partagees / possedees ailleurs    |

```

1  -- Table managed
2  CREATE TABLE cat.silver.clients (id INT, nom STRING);
3
4  -- Table external (LOCATION explicite)
5  CREATE TABLE cat.silver.ext_clients (id INT)
6  LOCATION 's3://bucket/clients/';

```

Listing 6 – Creation et conversion de tables

## 8.2 Controle d'accès : GRANT / REVOKE / DENY

Appliques a des **principals** (users, groups, service principals) aux bons niveaux de la hierarchie (catalog / schema / table).

```

1  GRANT SELECT ON TABLE cat.silver.clients TO 'analystes';
2  GRANT USAGE ON SCHEMA cat.silver TO 'analystes';
3  REVOKE SELECT ON TABLE cat.silver.clients FROM 'stagiaires';
4  DENY MODIFY ON TABLE cat.silver.clients TO 'lecture_seule';

```

Listing 7 – Privileges Unity Catalog

## 8.3 Masquage et securite au niveau ligne/colonne

- **Column masking** : masquer une colonne sensible selon le groupe de l'utilisateur.
- **Row-level security (RLS)** : filtrer les lignes visibles selon l'utilisateur.
- **ABAC policies** (Attribute-Based Access Control) : controler centralement le filtrage de lignes et le masquage de colonnes pour les donnees sensibles, via des attributs.

### ! Piège d'examen

**Column masking** agit sur les *colonnes* (cache une valeur). **Row-level security** agit sur les *lignes* (cache des enregistrements). **ABAC** = gestion centralisee des deux via des attributs/tags, a l'echelle.

### QCM

On veut que les analystes voient la table clients mais que le numero de carte bancaire soit masque pour les groupes non autorises. Quel mecanisme ?

A. Row-level security B. **Column-level masking** C. REVOKE sur la table D. Table external

Reponse : **B**. Le masquage de colonne cache une valeur precise selon le groupe, sans retirer l'accès a toute la table.

## 9. Checklist hands-on (a savoir faire les yeux fermes)

Le guide de preparation insiste : **lire ne suffit pas**. Construis chacune de ces taches dans Databricks Free Edition (< 30 min chacune). Toute tache que tu ne sais pas faire = ta prochaine cible de revision.

| Tache                                                                                                       | Objectif |
|-------------------------------------------------------------------------------------------------------------|----------|
| <input type="checkbox"/> Ingerer des fichiers d'un dossier cloud avec <b>Auto Loader</b> (schema evolution) | §2       |

| Tache                                                                                                                   | Objectif |
|-------------------------------------------------------------------------------------------------------------------------|----------|
| <input type="checkbox"/> Charger des CSV avec <b>COPY INTO</b> dans une table UC                                        | §2       |
| <input type="checkbox"/> Nettoyer bronze → silver (nulls, types, <b>dropDuplicates</b> )                                | §3       |
| <input type="checkbox"/> Faire un <b>broadcast join</b> + agregations ( <b>approx_count_distinct</b> )                  | §3       |
| <input type="checkbox"/> Creer une <b>materialized view</b> et une <b>streaming table</b>                               | §3       |
| <input type="checkbox"/> Construire un <b>Lakeflow Job</b> multi-taches avec dependances + retries                      | §4       |
| <input type="checkbox"/> Configurer un trigger <b>file arrival</b> et un trigger <b>scheduled</b>                       | §4       |
| <input type="checkbox"/> Deployer un <b>Automation Bundle</b> sur une cible via la <b>CLI</b>                           | §5       |
| <input type="checkbox"/> Lire la <b>Spark UI</b> et identifier un <b>data skew</b>                                      | §6       |
| <input type="checkbox"/> Appliquer <b>GRANT</b> / <b>REVOKE</b> et un <b>column mask</b> en SQL                         | §7       |
| <input type="checkbox"/> Creer une table <b>managed</b> et une <b>external</b> , puis les DROP (observer la difference) | §7       |

## 10. Les 5 questions officielles du guide (corrigees)

Ces questions proviennent du guide officiel. Comprends *pourquoi* chaque reponse est correcte.

1. **Data skew (Spark UI).** Tache 10 min vs 30s, Max shuffle read  $\gg$  median. **Reponse B** : AQE + skew join handling decoupe la partition au runtime.
2. **Compute / plateforme.** Iteration rapide, rollback, audit, source unique IA+BI. **Reponse B** : Delta Lake (ACID + time travel) gouverne par Unity Catalog.
3. **Audit logs.** Format et comportement des logs d'audit Databricks. **Reponse A** : JSON, < 15 min, les livraisons peuvent ecraser des fichiers.
4. **Cluster pour analystes SQL ad hoc.** Concurrence + demarrage rapide + cout. **Reponse C** : cluster high-concurrency avec autoscaling.
5. **CI/CD pipelines ETL modulaires.** Deployer, versionner, orchestrer. **Reponse D** : Automation Bundles (DABs), versionnes dans Git, promus par CI/CD.

## Annexe : memo derniere minute (a relire 1h avant)

### A retenir : les reflexes qui font gagner des points

- **Noms actuels** : Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, Git Folders, Automation Bundles, Lakeflow Connect. Jamais Workflows / DLT / Repos / DAB dans une bonne reponse.
- **Auto Loader** = streaming, gros volumes, schema evolution auto. **COPY INTO** = batch SQL idempotent. **Lakeflow Connect** = sources d'entreprise managees.
- **Delta** = ACID + time travel (rollback/audit). **Unity Catalog** = gouvernance, hierarchie catalog → schema → objet.
- **Medaillon** : bronze (brut) → silver (propre) → gold (agregé BI).
- **View** recalculee / **Materialized view** precalculee / **Streaming table** incrementale.

- **Data skew** → AQE + skew join handling.
- **Liquid Clustering** (remplace partition+ZORDER) ; **Predictive Optimization** (OPTIMIZE/VACUUM auto).
- **Triggers** : scheduled (temps) vs file arrival / table update (data-driven).
- **Bundles + Git + CLI** = CI/CD. **validate** → **deploy** → **run**.
- **Managed** : DROP supprime les donnees. **External** : DROP garde les donnees.
- **GRANT/REVOKE/DENY** sur principals ; **column mask** (colonne) / **RLS** (ligne) / **ABAC** (centralise par attributs).

Verifie toujours sur <https://docs.databricks.com> – bonne chance !